

CPS188: Lab 04 - Repetition and Loops

Name: Iqraa Wahid

Student Number: 501287410

Problem #1:

Algorithm:

1. Determine the number of rows
2. Add appropriate spacing before printing the pascal triangle
3. Determine columns
4. Calculate and output the coefficients of the pascal triangle

Code:

```
#include <stdio.h>

int main (void){
    int tot_Rows=9;//pascal triangle has 9 rows
    for(int rows=0;rows<tot_Rows;rows++){
        int num=1;//number that is outputted in the pascal triangle
        //adds appropriate number of spaces to center the pascal
triangle
        for(int space=0;space<tot_Rows-rows;space++){
            printf(" ");
        }
        //number outputted: where j is the number of columns
        for(int columns=0;columns<=rows;columns++){
            printf("%4d", num);
            num = num * (rows - columns) / (columns + 1);
        }
        printf("\n");//adds a new line
    }
    return (0);
}
```

Output:

```
    1
   1 1
  1 2 1
 1 3 3 1
1 4 6 4 1
1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
```

1 8 28 56 70 56 28 8 1

Screenshot:

Output Clear

```

      1
     1 1
    1 2 1
   1 3 3 1
  1 4 6 4 1
 1 5 10 10 5 1
1 6 15 20 15 6 1
1 7 21 35 35 21 7 1
1 8 28 56 70 56 28 8 1

=== Code Execution Successful ===
```

Problem #2:

Algorithm:

1. Import the data from the file to the program for each employee
2. Calculate the gross pay of each employee according to their total number of hours worked
3. Output the gross pay for each employee

Code:

```
#include <stdio.h>
```

```
double calculate_gross_pay(int total_hours, double wage){
    double gross_pay;
    double premium;
    if(total_hours<=15){
        gross_pay=total_hours*wage;
    }
    else if(15<total_hours && total_hours<=25){
        premium=0.05*total_hours*wage;
        gross_pay=(total_hours*wage)+premium;
    }
    else{
        if(total_hours>25){
            premium=0.10*total_hours*wage;
```

```

        gross_pay=(total_hours*wage)+premium;
    }
}
return gross_pay;
}

int main (void){
    printf("Columnar Report:\n\n");
    printf("%s %17s %15s\n", "Employee Number:", "Total Hours:", "Gross Pay:");
    int emp_num, shift; //declaring employee number and shift variables
    int total_hours, hours;
    double wage, gross_pay; //declaring base wage rate variable
    FILE *myfile; //declaring file
    myfile=fopen("data.txt", "r"); // opening file to reading mode

    fscanf(myfile, "%d", &emp_num); //storing the first element in the file in variable emp_num

    while(!feof(myfile)){//keep reading until all elements have been reading
        fscanf(myfile, "%d", &shift); //storing the second element in the file in variable shift
        fscanf(myfile, "%lf", &wage); //storing the third element in the file in variable wage
        total_hours=0.0;
        for(int i=0;i<shift;i++){
            fscanf(myfile,"%d", &hours);
            total_hours=total_hours+hours;
        }
        gross_pay=calculate_gross_pay(total_hours, wage);

        printf("%d %18d %14s $%0.2lf\n", emp_num, total_hours, " ", gross_pay);

        //updating the value for the next employee:
        fscanf(myfile, "%d", &emp_num);
    }
    fclose(myfile);
}

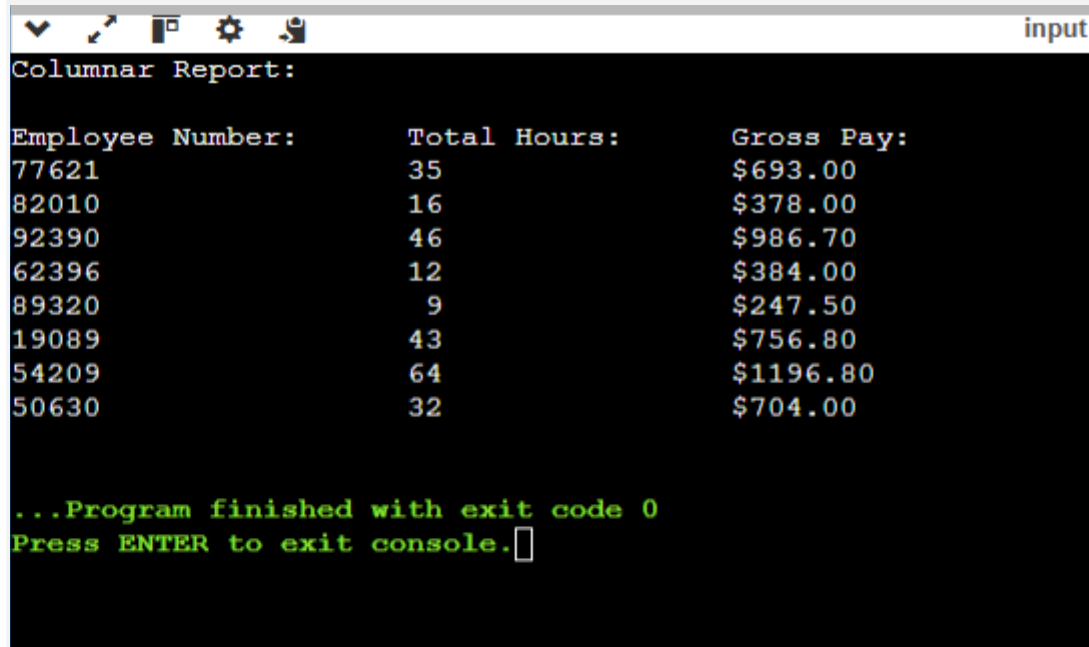
```

Output:

Columnar Report:

Employee Number:	Total Hours:	Gross Pay:
77621	35	\$693.00
82010	16	\$378.00
92390	46	\$986.70
62396	12	\$384.00
89320	9	\$247.50
19089	43	\$756.80
54209	64	\$1196.80
50630	32	\$704.00

Screenshot:



```
input
Columnar Report:

Employee Number:      Total Hours:      Gross Pay:
77621                 35                $693.00
82010                 16                $378.00
92390                 46                $986.70
62396                 12                $384.00
89320                 9                 $247.50
19089                 43                $756.80
54209                 64                $1196.80
50630                 32                $704.00

...Program finished with exit code 0
Press ENTER to exit console.█
```

By Hand Calculations (for specific test data given):

77621 6 18.00 6 7 5 2 7 8
⇒ Total Hours: 35; Gross Pay = $(0.10 * 18.00 * 35) + (35 * 18.00) = \693.00

82010 3 22.50 7 3 6
⇒ Total Hours: 16; Gross Pay = $(0.05 * 22.50 * 16) + (16 * 22.50) = \378.00

92390 8 19.50 4 8 4 7 2 7 8 6
⇒ Total Hours: 46; Gross Pay = $(0.10 * 19.50 * 46) + (46 * 19.50) = \986.70

62396 2 32.00 6 6
⇒ Total Hours: 12; Gross Pay = $32.00 * 12 = \$384.00$

89320 1 27.50 9
⇒ Total Hours: 9; Gross Pay = $9 * 27.50 = \$247.50$

19089 7 16.00 5 6 7 8 9 2 6
⇒ Total Hours: 43; Gross Pay = $(0.10 * 16.00 * 43) + (43 * 16.00) = \756.80

54209 11 17.00 8 9 2 5 8 9 2 5 6 8 2
⇒ Total Hours: 64; Gross Pay = $(0.10 * 17.00 * 64) + (64 * 17.00) = \1196.80

50630 4 20.00 8 8 8 8
⇒ Total Hours: 32; Gross Pay = $(0.10 * 20.00 * 32) + (32 * 20.00) = \704.00

Problem #3:

Algorithm:

1. Calculate and output the maximum temperature the cylinder could have before bursting
2. Start from 0 degrees celsius and increase by 250 degrees celsius each time, outputting the temperature in celsius and kelvins, and pressure in atm until the cylinder explodes
3. Once the cylinder explodes, output "KABOOM!" and the temperature and pressure it must surpass for the cylinder to explode

Code:

```
#include <stdio.h>
```

```
int main (void){
    //a:
    int p1=50, t1=300, p2=500, t2;
    t2=(p2*t1)/p1; //solving for max. temperature
    printf("The cylinder can withstand %d°K temperature before bursting.\n\n", t2);

    //b:
    double tempK, pressure, k=1.0;
    printf("%-20s %-20s %-20s\n", "Temperature (°C)", "Temperature (°K)", "Pressure (atm)");
    printf("%-20s %-20s %-20s\n", "-----", "-----", "-----");
    for(double tempC=0.0;tempC<=(t2-273.15);tempC=tempC+250){
        tempK=tempC+273.15;
        pressure=(p1*tempK)/t1;
        printf("%-20.2lf %-20.2lf %-20.3lf\n", tempC, tempK, pressure);
    }
    printf(">=2726.85 %9s >=3000.00 %10s >=500.00\n", " ", " ");
    printf("KABOOM!");
}
```

Output:

The cylinder can withstand 3000°K temperature before bursting.

Temperature (°C)	Temperature (°K)	Pressure (atm)
0.00	273.15	45.525
250.00	523.15	87.192
500.00	773.15	128.858
750.00	1023.15	170.525
1000.00	1273.15	212.192
1250.00	1523.15	253.858
1500.00	1773.15	295.525
1750.00	2023.15	337.192
2000.00	2273.15	378.858
2250.00	2523.15	420.525
2500.00	2773.15	462.192
>=2726.85	>=3000.00	>=500.00
KABOOM!		

Screenshot:

```
input
The cylinder can withstand 3000°K temperature before bursting.

Temperature (°C)      Temperature (°K)      Pressure (atm)
-----
0.00                  273.15                45.525
250.00                523.15                87.192
500.00                773.15                128.858
750.00                1023.15               170.525
1000.00               1273.15               212.192
1250.00               1523.15               253.858
1500.00               1773.15               295.525
1750.00               2023.15               337.192
2000.00               2273.15               378.858
2250.00               2523.15               420.525
2500.00               2773.15               462.192
>=2726.85             >=3000.00             >=500.00
KABOOM!

...Program finished with exit code 0
Press ENTER to exit console.
```